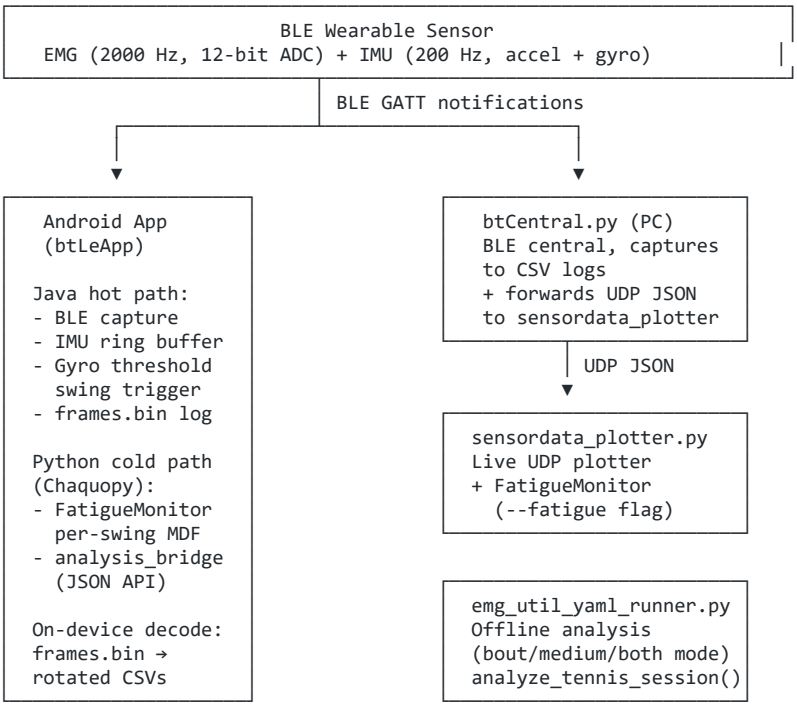
High-Level Design: EMG Fatigue Monitoring System

Overview

This system captures raw EMG + IMU data from a BLE wearable sensor, performs real-time fatigue monitoring during activity (tennis, FTC robotics, etc.), and supports deep offline analysis after a session. The pipeline spans three layers: Android app, PC live plotter, and PC offline analysis.

The core challenge is not signal processing per se — it is **making the pipeline robust to the practical realities of wearable sensors**: electrode contact loss, motion artifact, warm-up transients, and the difficulty of establishing a valid baseline in a noisy real-world environment. Without the design decisions documented in the "Algorithm Design" section below, MDF does not drop meaningfully even with valid sensor data.

System Architecture



Data Flow

Real-time path (Android)

```
BLE notification (EMG sample)
  → Java: append to emgFatigueBuf (ring buffer, 2600 samples)
  → Java: append to frames.bin (raw capture log)

BLE notification (IMU sample)
  → Java: check gyro magnitude >= threshold (300 dps)
  → if triggered AND debounce OK (2000ms):
      → snapshot emgFatigueBuf (2600 samples = 1300ms @ 2000Hz)
      → fatigueThread: call analysis_bridge.push_swing_realtime(emg_json, t_peak_ms, gmag)
          → Python: FatigueMonitor.push_swing()
              → preprocess_emg() [bandpass + notch]
              → mnf_mdf() [Welch PSD → MDF]
              → rms, env_mean
              → baseline collection / fatigue delta
          → return JSON: {mdf, rms, mdf_drop_pct, fatigue_level, warning, ...}
      → Java: update UI overlay
```

Real-time path (PC live plotter)

```
btCentral.py → UDP JSON → sensordata_plotter.py
  → raw/env/imu packets → rolling deque buffers → matplotlib animation

  IMU packet (gmag >= --imu-thr-dps, 2000ms debounce):
    → snapshot emg_fatigue_buf (deque, maxlen=2600)
    → fatigue_q.put_nowait((snap, t_peak_ms, gmag))
    → fatigue_worker thread: FatigueMonitor.push_swing()
    → update MDF trend subplot + annotation
```

Offline path (PC)

```
frames.bin (Android)
  → btcentral_offline_decode.py: decode_frames_bin_to_rotated_csv()
  → emg_log_<session_id>_<open_id>_<idx>.csv
  → imu_log_<session_id>_<open_id>_<idx>.csv

emg_util_yaml_runner.py --config config.yaml --log-dir <folder>
  → capture_loader.discover_sessions()
  → load_session_stitched() [stitch all BLE captures as one tennis session]
  → detect_swings_from_imu() [fixed 300 dps threshold → delegates to core.detect_swing_windows_from_imu()]
  → activity_segments_from_imu() [percentile hysteresis gating]
  → per_swing_emg_features_stitched() [full: MDF+RMS+MAV+WL+ZC+SSC+MNF]
     OR per_swing_emg_medium_stitched() [medium: envelope+MDF+RMS only]
     OR segment_bouts_from_active() [bout: IMU only, no EMG]
  → baseline selection + fatigue deltas
  → plots + session_summary.json
```

Key Files

File	Location	Purpose
`emg_util.py`	`emg_util/`	Core library: preprocessing, features, swing detection, offline analysis
`fatigue_monitor.py`	`emg_util/`	**Canonical** `FatigueMonitor` + `preprocess_emg` (pandas-free, Android-compatible)
`emg_util_yaml_runner.py`	`emg_util/`	YAML-driven CLI: offline tennis session analysis
`capture_loader.py`	`emg_util/`	Session discovery, CSV loading, stitching
`capture_health.py`	`emg_util/`	EMG signal health check (contact loss detection)
`config.yaml`	`emg_util/`	Full analysis config

File	Location	Purpose
`config_fast.yaml`	`emg_util/`	Fast/medium mode config
`emg_util_nopandas.py`	`btLeApp/.../python/`	Pandas-free subset of emg_util.py for Android (Chaquopy)
`analysis_bridge.py`	`btLeApp/.../python/`	Python↔Java API: reset*fatigue*monitor, push*swing*realtime, get*fatigue*summary
`btCentral.py`	`py_scripts/`	PC BLE central: capture to CSV + UDP forwarding
`sensordata_plotter.py`	`py_scripts/`	Live UDP plotter + real-time FatigueMonitor

Analysis Modes (offline, `emg_util_yaml_runner.py`)

Configured via `features.mode` in YAML:

Mode	Speed	What it computes
`bout`	Fastest	IMU swing detection + bout segmentation only. No EMG feature extraction.
`medium`	Fast	Envelope + MDF + RMS per swing + windowed RMS. Skips MAV/WL/ZC/SSC/MNF.
`both` (default)	Full	All features: MDF + RMS + MAV + WL + ZC + SSC + MNF per swing + windowed features + bout summary.

Algorithm Design: Key Contributions and Rationale

This section documents the design decisions that make the fatigue pipeline work in practice. Each subsection describes a real problem encountered with real sensor data and the solution adopted.

---

1. EMG Signal Preprocessing

Problem: Raw MyoWare 2.0 ADC output is a 12-bit unsigned integer (0–4095) biased around ~1830–1843 (not the theoretical 2048). It contains:

- DC offset (~1840 counts)
- Motion artifact (0–20 Hz)
- Power line interference (60 Hz + harmonics)
- Useful EMG signal (20–500 Hz, hardware-filtered by MyoWare)

Without preprocessing, `abs(raw)` ≈ 1840 at rest (the DC level), making the software envelope useless and MDF computation dominated by the DC spike at 0 Hz.

Solution (`preprocess_emg()` in `fatigue_monitor.py`):

```
raw (0–4095 ADC)
  → scipy.signal.detrend()          [remove linear trend / DC]
  → butter(4, [50, 450], 'bandpass') [50–450 Hz bandpass]
  → iirnotch(60 Hz) + iirnotch(120 Hz) [power line rejection]
  → output: zero-mean AC signal, ~±100–500 counts during contraction
```

Why 50 Hz low cutoff (not 20 Hz):

- MyoWare hardware already bandpasses at 20–500 Hz
- Software 50 Hz cutoff additionally rejects the 20–50 Hz band (residual motion artifact)
- MDF spectral estimation is more accurate with motion artifact removed
- The 20–50 Hz band contains little useful fatigue information for surface EMG

Why this is the single source of truth: `fatigue_monitor.py` is the canonical implementation. `emg_util.py` and `emg_util_nopandas.py` both import from it. Previously three copies existed with diverging parameters (20 Hz vs 50 Hz low cutoff) — the Android path was silently using a different filter than the PC path.

DC removal for envelope computation: The software envelope (`ENV_sw`) computes `abs(raw - DC)` where DC is tracked by a slow LPF (0.1 Hz). Without DC removal, `abs(raw)` ≈ 1840 at rest → the LPF converges to 1840 → the envelope appears as a flat line near the top of the plot, making HW vs SW envelope comparison meaningless.

---

2. Swing Detection: Why Fixed Threshold (Not Z-Score)

Problem: The original swing detector used a z-score approach: compute mean and std of gyro magnitude during the first 2 seconds (baseline), then flag samples where z-score > threshold.

This failed in practice for two reasons:

1. **Player already moving at capture start.** If the player starts swinging immediately, the "baseline" period contains swing peaks. The mean and std are inflated, making the z-score threshold too high — real swings are missed.
2. **Rest floor too low for wrist sensors.** At rest, wrist gyro magnitude ≈ 5–30 dps. A tennis forehand peaks at 400–800 dps. The z-score threshold (e.g., z > 5) would require `mean + 5×std > 300 dps`, which means `std > (300 - 30) / 5 = 54 dps`. This is only achievable if the baseline period itself contains significant movement — a circular dependency.

Solution: Fixed 300 dps threshold

```
mag_dps = sqrt(gx² + gy² + gz²)  # gyro magnitude in dps
above = mag_dps >= 300.0          # fixed threshold
```

Why 300 dps:

- Tennis forehand/backhand peak gyro: 400–800 dps (well above threshold)
- Walking arm swing: 100–200 dps (well below threshold)
- 300 dps sits cleanly between the two distributions with ~100 dps margin on each side
- Consistent across `ThresholdSwingDetector`, `HysteresisSwingDetector`, offline pipeline, and Android Java (after Phase 3 fix)

Why 2000ms minimum gap (debounce):

- Minimum realistic inter-swing interval during a rally: ~1.5–2 seconds
- 1000ms debounce allowed double-counting: backswing (peak ~200ms before contact) + forward swing (peak at contact) both exceeded threshold within 1 second
- 2000ms ensures only one detection per stroke

Window merge step: After peak detection and min-gap enforcement, overlapping `[t_start, t_end]` windows are merged. This handles the case where two peaks are separated by exactly `min_gap_ms` but their windows overlap (e.g., `pre_ms=250, post_ms=450, min_gap_ms=2000` → windows are 700ms wide, no overlap at 2000ms gap). The merge step is a correctness guarantee for edge cases.

---

3. IMU Activity Gating: Separating Rallies from Rest

Problem: A tennis session contains both active periods (rallies, ~30–60% of session time) and rest periods (between points, changeovers, ~40–70% of session time). During rest:

- EMG signal is noise-dominated (flat spectrum, MDF ≈ 100 Hz)
- IMU gyro is near zero (no swings)
- Windowed MDF features are meaningless

If rest-period windows are included in the fatigue trend, the MDF appears to *increase* over time (rest at start → active play at end), giving a negative MDF drop % — the opposite of fatigue.

Solution: Percentile hysteresis activity gating (`activity_segments_from_imu()`)

```
1. Compute hybrid IMU intensity: I = α × gyro_mag_dps + (1-α) × acc_dyn_g
   where acc_dyn = |acc_mag - 1g| (dynamic acceleration, gravity-removed)
   EMA-smoothed with τ = 500ms to bridge intra-rally gaps

2. Compute session-wide percentiles of I:
   - p_low  = 20th percentile (rest floor)
   - p_high = 70th percentile (active play level)

3. Hysteresis state machine:
   - Enter ACTIVE when I > p_high (sustained activity)
   - Exit ACTIVE when I < p_low (sustained rest)
   - Minimum active segment duration: 5 seconds
   - Merge segments within 3 seconds of each other

4. Output: list of (t_start_ms, t_end_ms) active segments
```

Why percentile hysteresis (not fixed threshold):

- Session intensity varies by player, sensor placement, and activity type
- A fixed threshold (e.g., 200 dps) would miss low-intensity players or misclassify high-intensity rest (e.g., walking between points)
- Percentile-based threshold adapts to the session's own intensity distribution
- Hysteresis prevents rapid toggling at the boundary

Effect on baseline selection: `select_baseline_swings()` prefers swings with `in_active == 1` (within active segments). This prevents rest-period swings (lower MDF due to noise) from contaminating the baseline.

---

4. Baseline Establishment: The Core Challenge

Problem: The fatigue metric is `mdf_drop_pct = (baseline_mdf - current_mdf) / baseline_mdf × 100`. If `baseline_mdf` is wrong, every subsequent swing's fatigue estimate is wrong.

Two failure modes observed with real data:

Failure mode A — Poor electrode contact at session start: The first BLE connection (≈ t = 0–500 s) often has near-zero EMG RMS — the electrode is not yet in good contact with the skin. The IMU-based swing detector still fires (gyro threshold is independent of EMG quality). Those swing windows contain mostly broadband noise:

- Noise spectrum is flat → MDF ≈ 100 Hz (Nyquist/2 of the noise floor)
- Real muscle EMG during active play → MDF ≈ 160–200 Hz

If the first 20 swings are noise-dominated, `baseline_mdf ≈ 100 Hz`. Every subsequent real swing computes `(100 − 200) / 100 × 100 = −100%` — the opposite of fatigue.

Failure mode B — Warm-up period: Even with good electrode contact, the first 5–10 minutes of a tennis session are warm-up: lower muscle temperature → slower conduction velocity → lower MDF (~120 Hz vs ~180 Hz at full warm-up). Using warm-up swings as baseline anchors the reference too low.

Solution: Three-layer baseline protection

Layer 1 — Warmup skip (`warmup_skip_min = 5.0`)

```
cutoff_ms = session_t0_ms + warmup_skip_min * 60 * 1000
baseline_candidates = swings[swings.t_peak_ms >= cutoff_ms]
```

Skips the first 5 minutes entirely. Swings before this cutoff are excluded from baseline selection and from the fatigue trend plot (shown as a separate "pre-warmup" region).

Layer 2 — RMS quality filter

```
# Offline: session-wide reference
session_p90_rms = np.percentile(all_swing_rms, 90)
rms_threshold = 0.25 * session_p90_rms
valid = baseline_candidates[baseline_candidates.rms >= rms_threshold]

# Real-time (FatigueMonitor): candidate pool reference
candidate_p90_rms = np.percentile(collected_rms, 90)
rms_threshold = 0.25 * candidate_p90_rms
valid = [s for s in candidates if s.rms >= rms_threshold]
```

Why RMS is the right quality proxy:

- Noise-dominated windows: RMS ≈ 5–20 (broadband noise floor)
- Real muscle contraction: RMS ≈ 50–500 (depends on effort)
- The ratio is 10–100×, making RMS a reliable discriminator
- MDF alone cannot distinguish noise from real EMG (both can have similar frequency content)

Why 90th percentile as reference (not mean or median):

- If >50% of swings are noise-dominated (poor contact throughout), the median RMS is at the noise floor → threshold = 0.25 × noise*floor* ≈ *noise*floor → filter does nothing
- The 90th percentile represents the "best effort" swings — even if 80% of swings are noise, the top 10% are likely real muscle contractions
- 0.25 × p90 is a conservative threshold: a swing must have at least 25% of peak-effort RMS to be considered valid for baseline

Why 3× hard cap in real-time: In streaming mode, the session-wide p90 is not available. The adaptive filter collects up to `3 × baseline_swings` candidates before finalizing. If the first 20 candidates are all noise-dominated, their p90 is still noise-level and the filter does nothing. By collecting up to 60 candidates, electrode contact typically improves and later candidates raise the p90, allowing the filter to correctly exclude the early noise-dominated swings.

Layer 3 — 75th percentile for baseline MDF (not median)

```
baseline_mdf = np.percentile(valid_swing_mdf_values, 75)
```

Why 75th percentile (not median): Even after warmup skip and RMS filter, some baseline swings may be sub-maximal (player still warming up, or low-effort practice swings). Fresh muscles have the HIGHEST MDF. Using the median underestimates the baseline when the distribution is left-skewed (a few high-MDF swings and many medium-MDF swings). The 75th percentile better represents the "fresh state" MDF.

For RMS (effort proxy), the median is used — fatigue increases RMS, so the baseline should be the low-effort (fresh) level, not the peak.

---

5. MDF as Fatigue Indicator: Why It Works and When It Fails

Why MDF drops with fatigue: Muscle fatigue causes:

1. Accumulation of metabolites (H^+, lactate) → slows action potential conduction velocity
2. Slower conduction velocity → lower frequency content in the EMG signal
3. MDF (median frequency of the power spectrum) shifts downward: typically 10–25% drop over a fatiguing contraction

Why MDF is unreliable without the above pipeline:

Failure mode	Effect on MDF	Symptom
DC offset not removed	DC spike at 0 Hz dominates PSD	MDF ≈ 0–5 Hz (useless)
Motion artifact (0–20 Hz)	Low-frequency peak in PSD	MDF biased low
Noise-dominated window	Flat spectrum	MDF ≈ 100 Hz (Nyquist/2)
Warm-up swings in baseline	Low baseline MDF	Negative drop % (MDF "increased")
Rest-period windows in trend	MDF rises over session	Negative trend (opposite of fatigue)
Wrong FSR defaults	Gyro values 4× too small	Zero swings detected

Why Welch PSD (not plain FFT):

- Welch method averages multiple overlapping periodograms → reduces variance of PSD estimate
- Plain FFT PSD has high variance for short windows (1300ms = 2600 samples)
- Lower PSD variance → more stable MDF estimate → less noise in the fatigue trend
- `nperseg=1024, noverlap=512` gives 3 overlapping segments from 2600 samples

Minimum window size for reliable MDF:

- At 2000 Hz, 1300ms = 2600 samples
- Welch with nperseg=1024: frequency resolution = 2000/1024 ≈ 2 Hz
- MDF for surface EMG is typically 80–200 Hz → 2 Hz resolution is adequate
- Shorter windows (< 500ms) give frequency resolution > 4 Hz → MDF estimate too coarse

---

6. IMU-EMG Correlation and Effort Normalization

Problem: MDF drop % alone is an incomplete fatigue indicator because it conflates:

- True fatigue (metabolic, conduction velocity decrease)
- Effort variation (player swings harder in some rallies than others)
- Electrode contact variation (RMS changes with contact quality)

A player who swings harder in the second half of a session will show higher RMS and potentially higher MDF (more motor unit recruitment) — the opposite of fatigue in the MDF metric.

Solution: IMU dynamic acceleration as effort proxy

```
acc_dyn = |acc_mag - 1g|   # remove gravity component
imu_mean_acc_dyn = mean(acc_dyn) within swing window
```

```
acc_drop_pct = (baseline_acc_dyn - current_acc_dyn) / baseline_acc_dyn × 100
```

- `acc_drop_pct > 0`: player is swinging with less force (fatigue or reduced effort)
- `acc_drop_pct < 0`: player is swinging harder (warm-up or increased effort)

Why dynamic acceleration (not total acceleration):

- Total `acc_mag` at rest ≈ 1g (gravity). Subtracting 1g gives the dynamic component.
- During a swing, `acc_dyn` peaks at 1–3g (FSR=4g range), providing good resolution.
- Gyro features are not used for effort normalization because gyro saturated at FSR=250 dps (every hard swing clipped at ±32767 LSB). FSR was raised to 1000 dps in firmware to unlock gyro-based features.

Combined fatigue interpretation:

MDF drop %	acc*drop*pct	Interpretation
+15%	+10%	Genuine fatigue: MDF dropped AND force decreased
+15%	−5%	Ambiguous: MDF dropped but player swinging harder
−5%	+10%	Effort decrease (rest/pacing), not fatigue
−5%	−10%	Player warming up / increasing effort

---

7. Swing Window Design

Problem: The EMG window used for MDF computation must be:

1. Long enough for reliable Welch PSD (≥ 500ms)
2. Centered on the swing peak (maximum muscle activation)
3. Not contaminated by adjacent swings

Solution: Asymmetric window with padding

```
t_start = t_peak - pre_ms   (default: 250ms before peak)
t_end   = t_peak + post_ms  (default: 450ms after peak)
window  = 700ms total

For MDF computation: additional ±300ms padding (mdf_pad_ms)
→ padded window = 1300ms = 2600 samples @ 2000Hz
```

Why asymmetric (250ms pre, 450ms post):

- EMG activation begins ~100–200ms before the gyro peak (muscle fires before peak rotation)
- The follow-through phase (after peak) contains sustained activation
- More post-peak time captures the full contraction

Why 300ms MDF padding:

- The Welch PSD needs ≥ 1024 samples for nperseg=1024
- 700ms window = 1400 samples — barely enough
- 1300ms padded window = 2600 samples — 3 overlapping Welch segments → stable PSD

Why the ring buffer is 2600 samples (not 700): The ring buffer is sized for the padded MDF window (1300ms), not the display window (700ms). This ensures the full padded window is always available when `push_swing()` is called.

---

8. Motion Artifact and Sweat: Identification and Mitigation

This section documents the practical interference sources that corrupt EMG signals during tennis and similar high-intensity activities, explains their signatures, and describes both current mitigations and potential future work.

8.1 Time-Domain Signatures

True EMG signal:

- Zero-mean after detrending (stochastic, symmetric around zero)
- Dense, high-frequency spikes (20–500 Hz)
- Amplitude correlates with muscle activation timing
- Onset/offset aligns with voluntary contraction, not with racket mechanics

Motion artifact:

- Large, slow deflections — step-like or ramp-like transients
- Bandwidth: 0–20 Hz (much slower than EMG)
- Often synchronized with mechanical events:
- Swing start (arm acceleration)
- Ball impact (impulsive deceleration)
- Racket deceleration (follow-through)
- **Key diagnostic:** overlay IMU acceleration on EMG — motion artifact peaks align with IMU acceleration peaks. True EMG does not.

Sweat effect:

- Lowers skin–electrode impedance → increases baseline amplitude (higher RMS at rest)
- Does NOT directly change frequency content
- But sweat + motion = amplified artifact: lower impedance makes electrode displacement easier, increasing artifact energy and changing its frequency content
- Sweat amplifies motion artifact; it does not replace it

8.2 Frequency-Domain Effects on MDF

Classic EMG fatigue theory: Fatigue → metabolite accumulation → slower muscle fiber conduction velocity → EMG spectrum shifts left → MDF decreases (typically 10–25% over a fatiguing contraction).

Motion artifact frequency injection: Motion artifact injects strong low-frequency energy (< 20–30 Hz) into the PSD. Depending on artifact dominance, this produces two opposite effects on MDF:

Case	Mechanism	MDF effect	Symptom
Case 1: Low-frequency dominance	Artifact energy >> EMG energy	MDF drops artificially	False fatigue detection
Case 2: Transient + rectification + windowing	Artifact redistributes power across spectrum	MDF increases	MDF "improves" during active play — opposite of fatigue

Case 2 is the more insidious failure mode and is what is observed in practice:

- A hard swing produces a large motion artifact transient
- The transient, after bandpass filtering, spreads energy into the 50–200 Hz band
- The Welch PSD sees elevated mid-frequency power → MDF shifts right
- Result: MDF appears to *increase* during the most active (most fatiguing) swings

This is why the 50 Hz high-pass cutoff alone is insufficient — motion artifact energy can alias into the EMG band after filtering if the artifact is large enough.

8.3 Current Mitigations (Implemented)

Mitigation	Where	Effect
50 Hz high-pass cutoff	`preprocess_emg()`	Removes most motion artifact (0–50 Hz)
IMU-gated swing trigger	All paths	Only processes windows where gyro > 300 dps (real muscle activation)
RMS quality filter	Baseline selection	Rejects noise-dominated windows (low RMS = no real contraction)
Asymmetric window (250ms pre, 450ms post)	Swing window	Avoids ball-impact transient at t=0 (impact is at peak, not in pre-window)
Activity gating	Offline only	Excludes rest-period windows where motion artifact dominates
Warmup skip	All paths	Avoids early-session artifact from electrode settling

Limitation of current approach: The 50 Hz cutoff removes the bulk of motion artifact but cannot remove artifact that has been aliased or rectified into the 50–450 Hz band by the sensor's hardware amplifier. The MyoWare 2.0 hardware bandpass (20–500 Hz) passes motion

artifact in the 20–50 Hz band before the software filter sees it. Large transients (ball impact) can saturate the ADC briefly, creating broadband noise that survives the 50 Hz cutoff.

8.4 Potential Future Work

A. IMU-based artifact detection and gating

The strongest available signal for motion artifact is the IMU itself. Proposed approach:

```
1. Compute IMU acceleration magnitude: acc_mag = sqrt(ax² + ay² + az²)
2. Compute acc_dyn = |acc_mag - 1g| (dynamic component)
3. Identify artifact windows: acc_dyn > artifact_thr_g (e.g., 2.0g)
   AND the spike is impulsive (duration < 50ms) — ball impact signature
4. Mask those samples in the EMG window before MDF computation
   OR exclude the entire swing window if artifact fraction > 20%
```

This directly exploits the time-domain alignment between IMU peaks and motion artifact. The IMU is already captured synchronously with EMG — no additional hardware needed.

B. Adaptive notch / regression artifact removal

For sustained motion artifact (not just impulsive), a regression approach can work:

```
1. Low-pass filter IMU acceleration at 20 Hz → artifact_proxy(t)
2. Regress EMG(t) on artifact_proxy(t): EMG_clean = EMG - β × artifact_proxy
3. β estimated per-swing window using least squares
```

This is the standard approach in clinical EMG (e.g., Clancy & Hogan 1994). The key assumption is that motion artifact is linearly correlated with IMU acceleration — valid for electrode displacement artifact but not for impact transients.

C. Wavelet-based artifact separation

Motion artifact and EMG occupy different frequency bands AND different time scales. Discrete wavelet transform (DWT) can separate them:

```
1. DWT decompose EMG into approximation (low-freq) + detail (high-freq) coefficients
2. Threshold/zero the approximation coefficients (motion artifact lives here)
3. Reconstruct from detail coefficients only
```

Advantage over FIR/IIR filtering: wavelet thresholding is adaptive per-window and handles non-stationary artifacts better than a fixed-cutoff filter. Disadvantage: adds complexity; the 50 Hz FIR cutoff already handles most cases.

D. Sweat-aware impedance monitoring

Sweat lowers electrode impedance, which increases EMG amplitude but also makes the electrode more susceptible to displacement. A practical mitigation:

```
1. Monitor RMS trend over the session
2. If RMS increases monotonically (not correlated with swing intensity), flag as
   possible sweat-induced impedance change
3. Normalize MDF computation by RMS to partially decouple amplitude from frequency
```

Note: RMS normalization is already partially implemented via `acc_drop_pct` (effort normalization). A direct RMS trend monitor would complement this.

E. Ball-impact window exclusion

Ball impact produces the largest impulsive artifact. The impact occurs at the gyro peak (t*peak*ms). The current asymmetric window (250ms pre, 450ms post) includes the impact.

Proposed: exclude a ±20ms window around t*peak*ms from MDF computation:

```
# Mask impact transient before Welch PSD
mask = np.ones(len(seg), dtype=bool)
impact_i = np.searchsorted(ts, t_peak_ms)
mask[max(0, impact_i-40):impact_i+40] = False  # ±20ms @ 2000Hz = ±40 samples
seg_clean = seg[mask]
mnf_val, mdf_val = mnf_mdf(seg_clean, emg_fs_hz)
```

This is low-cost and directly targets the highest-energy artifact source.

F. Cross-session electrode contact quality score

Currently, the RMS filter is binary (above/below threshold). A continuous quality score would enable better baseline weighting:

```
quality_score = min(1.0, rms / session_p90_rms)
baseline_mdf = weighted_percentile(mdf_values, weights=quality_scores, p=75)
```

Higher-quality swings contribute more to the baseline. This is more robust than the binary filter when electrode contact degrades gradually over a session.

---

9. ENV-Based Activation Gating (Sports EMG Best Practice)

Problem: Even with IMU-based swing detection and EMG health filtering, some detected "swings" occur during rest periods or low-effort movements where the muscle is not actively contracting. These windows contain mostly noise and produce unreliable MDF values:

- Rest-period noise: MDF ≈ 100 Hz (flat spectrum, Nyquist/2 of noise floor)
- Active muscle contraction: MDF ≈ 150–200 Hz (real EMG frequency content)

Including rest-period windows in MDF/MNF computation adds scatter to the fatigue trend and can bias the baseline if rest swings are selected.

Solution: ENV (envelope) as activation proxy

The MyoWare 2.0 provides two EMG outputs:

- **RAW**: unprocessed EMG signal (20–500 Hz hardware bandpass)
- **ENV**: envelope output (rectified + low-passed at ~3.6 Hz)

ENV is not suitable for frequency analysis (MDF/MNF) because the low-pass filter removes the frequency content needed for spectral estimation. However, ENV is an excellent proxy for muscle activation intensity:

```
ENV high (> threshold) → muscle actively contracting → MDF/MNF valid
ENV low  (< threshold) → muscle at rest or noise-dominated → MDF/MNF unreliable
```

Implementation (`compute_env_gate_threshold()` + `apply_env_gate_to_swings()`):

```
# 1. Compute session-adaptive threshold from 1s ENV windows
win = int(1.0 * emg_fs_hz)  # 2000 samples @ 2000 Hz
n = len(env) // win
env_mean = np.mean(env[:n*win].reshape(n, win), axis=1)
threshold = np.percentile(env_mean, 75)  # upper quartile

# 2. For each swing: check if mean(ENV) >= threshold
for swing in swings:
    seg_env = env[swing.t_start:swing.t_end]
    env_mean = np.mean(seg_env)
    swing.env_active = 1 if env_mean >= threshold else 0

# 3. Filter swings by env_active before MDF/MNF computation
active_swings = [s for s in swings if s.env_active == 1]
```

Why percentile-based threshold (not absolute):

- ENV amplitude varies by electrode placement, skin impedance, and player physiology
- A fixed threshold (e.g., 1651 ADC units) would fail for different players or sessions
- The 75th percentile adapts to the session's own intensity distribution

- p75 (upper quartile) is conservative: keeps ~25% of time as "active"

Validation (log0305 real data):

- Correlation between ENV and RAW amplitude: **r = 0.696** (strong positive)
- ENV p10/p50/p90: 196 / 1223 / 2183 ADC units
- Suggested threshold (p75): **1651 ADC units**

This confirms ENV tracks muscle activation intensity and is suitable for gating.

Configuration (`config.yaml`):

```
env_gating:
  enable: false  # opt-in feature (default disabled)
  mode: percentile  # 'percentile' or 'absolute'
  percentile: 75  # p75 = upper quartile (conservative)
  absolute_threshold: 1651  # fixed threshold (only if mode='absolute')
  window_sec: 1.0  # window size for ENV mean computation
  apply_to: mdf_only  # 'mdf_only', 'baseline_only', or 'both'
```

When to use ENV gating:

- **Recommended:** Sessions with significant rest periods (tennis, interval training)
- **Not needed:** Continuous high-intensity activities (cycling time trial, rowing)
- **Caution:** If electrode contact is poor throughout, ENV may be noise-dominated and the percentile threshold will be at the noise floor → filter does nothing

Relationship to existing filters:

Filter	Purpose	When it runs
IMU swing detection (300 dps)	Detect movement events	Pre-filter (defines candidate swings)
EMG health filter	Reject contact-loss windows	Pre-filter (removes bad data)
RMS quality filter	Reject noise-dominated swings	Baseline selection only
ENV activation gate	Reject rest-period swings	MDF/MNF computation (optional)
Activity gating (IMU percentile)	Separate rallies from rest	Offline only

ENV gating complements the existing filters by adding a muscle-activation criterion. It is most effective when combined with IMU activity gating (offline) or used as a replacement for activity gating in real-time scenarios where session-wide IMU statistics are not available.

Sports EMG best practice pattern:

```
RAW (2000 Hz)  →  MDF/MNF (frequency analysis, fatigue)
ENV (3.6 Hz LP) →  Activation gating (muscle on/off detection)
IMU (200 Hz)    →  Motion artifact rejection (violent motion)
```

This three-signal approach is the recommended pattern for sports EMG:

- Use RAW for spectral features (MDF/MNF) — requires full frequency content
- Use ENV for amplitude/intensity features and gating — robust to sign changes
- Use IMU to reject artifact windows — independent motion reference

Implementation status:

- ✅ Configuration added (`config.yaml`)
- ✅ Core functions implemented (`emg_util.py`, `emg_util_nopandas.py`)
- ✅ Validated with real data (log0305: corr=0.696)
- ⏳ Integration into `emg_util_yaml_runner.py` (pending)
- ⏳ Testing with log0305 (after integration)

Real-time vs Offline Feature Comparison

Feature	FatigueMonitor (real-time)	medium mode (offline)	both mode (offline)
MDF	✅ Welch PSD	✅ Welch PSD	✅ Welch PSD

Feature	FatigueMonitor (real-time)	medium mode (offline)	both mode (offline)
RMS	✅	✅	✅
env_mean	✅ `nanmean(abs(raw))`	✅ `envelope()` LPF	✅ `envelope()` LPF
MAV	❌	❌	✅
WL	❌	❌	✅
ZC / SSC	❌	❌	✅
MNF	❌	❌	✅
Baseline warmup skip	✅ 5 min	✅ 5 min	✅ 5 min
Baseline RMS filter	✅ adaptive (3× cap)	✅ session p90	✅ session p90
Baseline MDF percentile	✅ 75th	✅ 75th	✅ 75th
mdf*drop*pct	✅	✅	✅
acc*drop*pct	✅	✅	✅
Activity gating	❌ (threshold trigger)	✅ percentile hysteresis	✅
Bout segmentation	❌	✅	✅

Note: `env_mean` means different things in real-time vs offline:

- Real-time: `nanmean(abs(raw))` — fast MAV proxy, no filtering
- Offline: output of `envelope()` (rectify + 2nd-order Butterworth LPF at 3.6 Hz) Column names differ: `env_mean` (FatigueMonitor) vs `emg_env_mean` (offline swing features).

Swing Detection: Real-time vs Offline

	Real-time Android (Java)	Real-time PC plotter	Offline (yaml_runner)
Method	Fixed threshold on gyro magnitude	Fixed threshold on gyro magnitude	Fixed threshold on gyro magnitude
Threshold	300 dps	300 dps (`ThresholdSwingDetector`)	300 dps (`thr_gyro_dps` config)
Debounce	2000ms	2000ms (`debounce_sec=2.0`)	2000ms (`min_gap_ms` config)
Implementation	Java hot path	`ThresholdSwingDetector` / `HysteresisSwingDetector`	`core.detect_swing_windows_from_imu(already_in_dps=True)`
Data format	Raw LSB (converted in Java)	dps (converted by plotter)	dps (converted by `load_session_stitched()`)
Window merge	N/A	N/A	✅ overlapping windows merged

All three paths now use 300 dps / 2000ms (aligned as of 2026-03-07). Previously Android used ~150 dps / 300ms, causing ~3× more false detections.

Python on Android: Performance Analysis

Why Python (Chaquopy) is acceptable for this use case

The key insight is that **Python is only called post-trigger, once per swing** (~every 1–3 seconds), not per-sample. Java owns the hot path.

Per-swing latency breakdown (mid-range Android, Snapdragon 6xx)

Operation	Estimated time
Chaquopy JNI call overhead	1–5 ms

Operation	Estimated time
`filtfilt` bandpass (2600 samples)	5–15 ms
`filtfilt` notch × 2	4–10 ms
`scipy.signal.welch` (nperseg=1024)	15–40 ms
RMS + env_mean	< 1 ms
Total per swing	**~25–70 ms**

Swing interval during active play: **1–3 seconds**. Processing runs in background thread → **no UI blocking**. Effective CPU overhead: **3–10% of swing interval** — completely acceptable.

Memory costs (one-time, loaded at app start)

Component	RAM
Python interpreter	~20–30 MB
NumPy	~20–30 MB
SciPy	~40–60 MB
Total	**~80–150 MB**

Startup cost: **2–6 seconds** cold start (one-time per app launch).

When to reconsider Java/Kotlin

Convert to Java only if:

1. Targeting devices with **< 2 GB RAM** (150 MB Python overhead is too much)
2. Need **< 20 ms** per-swing latency (e.g., real-time visual feedback mid-swing)
3. Hard APK size constraint (SciPy adds ~15–25 MB)

Quick win: drop SciPy dependency

`emg_util_nopandas.py` already has a **NumPy-only FFT fallback** for `mnf_mdf()` that activates when SciPy is absent. Dropping SciPy saves ~60–80 MB RAM and ~15–25 MB APK size. Tradeoff: plain FFT PSD vs Welch PSD — slightly less accurate MDF, acceptable for fatigue monitoring.

What to NEVER do

Never call Python per-sample at 2000 Hz. JNI overhead alone (~1–5 ms/call) would consume 2–10 seconds of CPU per second of data — completely unusable. The ring-buffer + post-trigger design correctly avoids this.

FatigueMonitor: Canonical Implementation

`FatigueMonitor` and `preprocess_emg` live in **`fatigue_monitor.py`** — the single source of truth:

File	Role
`emg_util/fatigue_monitor.py`	**Canonical** implementation (pandas-free, Android-compatible)
`emg_util/emg_util.py`	Re-exports `FatigueMonitor` and `preprocess_emg` via `from fatigue_monitor import ...`
`btLeApp/.../emg_util_nopandas.py`	Android copy — imports from `fatigue_monitor` submodule (git submodule)

`fatigue_monitor.py` is intentionally pandas-free and scipy-optional so it can run on Android via Chaquopy. Any change to `FatigueMonitor` or `preprocess_emg` is made **once** in `fatigue_monitor.py` and automatically available to all callers.

FatigueMonitor parameters

Parameter	Default	Description
`fs`	2000.0 Hz	EMG sampling rate
`warmup_skip_min`	5.0 min	Skip first N minutes before collecting baseline (use 0.0 for testing)
`baseline_swings`	20	Number of valid swings to establish baseline (after RMS filter)

Parameter	Default	Description
`mdf_warn_pct`	12.0%	MDF drop threshold for "moderate" fatigue warning
`mdf_high_pct`	20.0%	MDF drop threshold for "high" fatigue

Fatigue levels

Level	Condition
`warming_up`	In warmup period OR baseline not yet established
`fresh`	fatigue_pct < 33% (MDF drop < 6.6% of high threshold)
`moderate`	33% ≤ fatigue_pct < 67%
`high`	fatigue_pct ≥ 67% (MDF drop ≥ 13.4% of high threshold)

EMG window for push_swing()

- **Size**: 2600 samples = 1300 ms @ 2000 Hz
- **Includes**: ±300 ms padding around swing peak for PSD stability
- **Collected by**: Java ring buffer (Android) / Python deque (sensordata_plotter)

Known Issues / Maintenance Notes

1. **`warmup_skip_min` default differs** between `sensordata_plotter.py` (default=0.0, for testing) and `analysis_bridge.py` singleton (5.0, production). Use `--fatigue-warmup-min 5.0` for real sessions with the plotter.
2. **`imu_peak_mag` in `analysis_bridge.py`** is described as "acceleration magnitude" in the docstring but `sensordata_plotter.py` passes gyro magnitude (dps). The parameter is an effort proxy only and does not affect fatigue math, but the docstring is misleading.
3. **`env_mean` column name** means different things in real-time (`nanmean(abs(raw))`) vs offline (`emg_env_mean` from `envelope()` LPF). Do not compare these values directly.
4. **`_filter_baseline_by_rms` (offline) vs `FatigueMonitor` adaptive filter (real-time)** use different reference RMS values by design: offline uses session-wide p90 (all data available); real-time uses candidate pool p90 (best available approximation). Do not expect identical baseline MDF values between the two paths.
5. **Gyro saturation at FSR=250 dps** — old firmware clipped every hard swing at ±32767 LSB. Gyro-based effort features (`imu_peak_gyro_dps`) are unreliable for sessions captured before the FSR=1000 dps firmware update (2026-02-25). Use `imu_mean_acc_dyn` instead.

See also: `CHANGELOG.md` for history of changes, `SWING_DETECTION.md` for swing detector details.